

1.介绍一下数据库范式，我们在设计数据库时一定会遵守范式吗

一范式：字段是原子性不可分的

二范式：不允许有字段依赖于非主键的其他字段

三范式：消除传递依赖

BCNF：主键能决定所有非主键字段

不一定，有时候会故意违反范式来提高效率：比如im里面创建群聊，不是好友的不清楚对方id，从而无法快速获取对方信息，此时可以在群聊成员信息表中多存储该成员的部分信息，比如头像，昵称等等，而此时该字段在用户信息表中已经存在，我们这样做只是为了避免再进行一次表的查询，是违反范式提高效率

2.介绍一下数据库的锁（全局锁，表级锁，行级锁）

1.全局锁：面向全部事务和线程，锁住都不能用（一般用于数据库逻辑备份）

全局锁使用的例子（全局只读）：flush tables with read lock 解锁：unlock tables

2.表级锁

```
//表级别的共享锁，也就是读锁：  
//允许当前会话读取被锁定的表，但阻止其他会话对这些表进行写操作。  
lock tables t_student read;  
  
//表级别的独占锁，也就是写锁：  
//允许当前会话对表进行读写操作，但阻止其他会话对这些表进行任何操作（读或写）。  
lock tables t_stuent write;
```

- **表锁**：锁住整张表，写写互斥，读写互斥。

假如锁住这张表，让其只读，那么当前正在读的线程只能读这个表不能写，不能读别的表，别的线程也只能读，不能写

假如锁住让其只写，那么当前写线程可以读写，不能读写别的表，别的线程不能进行任何操作

- **元数据锁（MDL）**：主要为了防止做数据库基础操作时表结构被改动

当我们对数据库表进行读写时自动加MDL锁

进行CRUD操作时加MDL读锁（CRUD: 创建 读 更新 删除），对表结构做变更时加MDL写锁

MDL 是为了保证当用户对表执行 CRUD 操作时，防止其他线程对这个表结构做了变更。

当有线程在执行 select 语句（加 MDL 读锁）的期间，如果有其他线程要更改该表的结构（申请 MDL 写锁），那么将会被阻塞，直到执行完 select 语句（释放 MDL 读锁）。

反之，当有线程对表结构进行变更（加 MDL 写锁）的期间，如果有其他线程执行了 CRUD 操作（申请 MDL 读锁），那么就会被阻塞，直到表结构变更完成（释放 MDL 写锁）。

- **意向锁**（没什么好介绍的，是表级锁，因为一张表有很多数据，如果我对某一个数据加了行锁，就会同步给整张表上一个意向锁（表锁），后续如果还要对这个表上表锁，就直接看有没有意向锁，有则表示内部有数据被行锁占用，就拒绝本次表锁请求）

因为表锁与行锁互斥，有了表锁就不会上行锁

- **AUTO-INC锁**：对于主键自增（auto-increment）这种情况，插入数据时会加上 AUTO-INC锁进行整个表的锁定，插入后释放

3.行级锁

- **Record lock**

记录锁：锁住某一条记录（一行）：分为S型（share：读锁，共享）和X型（exclude，写锁，排他）

具体规则还是读写互斥，写写互斥那一套

- **Gap lock**

间隙锁，前面在可重复读隔离级别解决幻读中有提到，当前读时，直接锁住间隙，不让别的事务插入（但这个只是锁住间隙，不锁具体值）

假如有三个数据3,4,5 用间隙锁，只能保证3和4之间不插入个3.5进来，不能保证3不被修改为2

- **Next--Key lock**

临界锁，既锁数据，又锁空隙

- 插入意向锁

前面有个表级的意向锁，是用于判断表中是否上了行锁，**这里的插入意向锁，是为了判断是否该位置可以插入**，当插入操作发生时，会对插入的范围加一个插入意向锁，后面再来插入操作，判断有锁就会停止插入

3.锁（互斥，自旋，读写，乐观悲观）

- 盲等锁（自旋锁）：会一直等待，直到申请到锁

盲等锁底层：原子操作的置换且检测（TAS: test-and-set）

```
int TestAndSet(int *old_ptr, int new)
{
    int old = *old_ptr;
    *old_ptr = new;
    return old;
}
```

一个函数，将指针传入的值修改为新值并返回旧值

```
typedef struct lock_t{
    int flag;
} lock_t;

void init(lock_t *lock)
{
    lock->flag = 0;
}

void lock(lock_t *lock)
{
    while(TestAndSet(&lock->flag, 1) == 1)
        ; // do noting
}

void unlock(lock_t *lock)
{
    lock->flag = 0;
}
```

一把自旋锁，内部一个flag值，为0时标志没人用，为1时标志锁被锁定。

假如为0，那么调用上锁函数，会将flag置1，返回0，（旧值），因为返回值不为1，所以退出。

假如为1，那么调用上锁函数，flag本来就是1，返回1，一直循环

适用场景：多核处理器，时间片轮转比较快，拿到锁资源就能运行

- **互斥锁 (mutex)**：实现思路可以在循环判断的那个函数那里，如果申请锁失败，就保存TCB，把TCB插入等待队列，设置线程为等待状态并调度后续程序

互斥锁底层：维护了一个等待队列和一个状态字段，状态字段标志是否能上锁，如果不能上锁，却申请了，就把当前线程挂起在锁的等待队列上，直到能上锁后，从中取出一个线程上锁并运行

- **读写锁**：读写互斥，写写互斥。**用于读多写少的情景**
- **乐观锁和悲观锁**：悲观锁认为多线程同时修改资源冲突概率高，所以访问资源前必须上锁，乐观锁先修改资源，再验证期间有没有发生冲突，如果没有，则修改完成，否则放弃本次操作（乐观锁也叫无锁编程）

乐观锁底层采用**CAS思想**（compare and swap 比较并交换）：其思想为先记录要修改的变量的旧值，然后拷贝一个要修改变量，修改该变量的值，最后比较记录的旧值是否和现在的数据值相等，是则交换

比如我i=1，我记录1，想把他修改为6，那么我可以拷贝一个i作为临时变量，修改i=6，看此时真的i是不是还等于1，是则没人改，修改为6，否则我就放弃本次操作

4.给两个容器，一个容量x，一个容量y（假设y永远大于等于x，不大于时调换顺序），容器中没有水，假设你有无限多的水，如何获取一个装z升水的容器。你可以做以下操作

- 1.把一个容器中水全部倒掉
- 2.把一个容器中水装满
- 3.把一个容器中的水倒入另一个容器中，直到另一个容器装满或者自己的水倒完

比如两个容器容量分别有3升，5升，求取4升水。假设两个容器分别为A,B，我可以进行如下操作

- 操作1：B执行第二步，目前A (0) B (5)
- 操作2：B执行第三步，目前A (3) B (2)
- 操作3：A执行第一步，目前A (0) B (2)
- 操作4：B执行第三步，目前A (2) B (0)

- 操作5： B执行第二步，目前 A (2) B (5)
- 操作6： B执行第三步，目前 A (3) B (4) (成功获取4)

本质是找状态转移规律

状态	操作 (1, A) 代表A做了操作1	新状态	变化量
(0,0)	(2, A)	(x, 0)	+x
(0,0)	(2, B)	(0, y)	+y
(x, 0)	(2, B)	(x, y)	+y
(x, 0)	(3, A)	(0, x)	0
(0, y)	(3, B)	(x, y-x)	0
(0, x)	(2, B)	(0, y)	+y-x
(0, x)	(2, A)	(x, x)	+x
(x, y-x)	(1, A)	(0, y-x)	-x
(x, x)	(3, A)	(0,2x) 或 (x- (y-x) , y) 即 (2x-y, y)	0

每次操作产生的变化量都是 $ax+by$ ，即我们只要找到 $ax+by=z$ ，（a, b为整数即可），首先可以类似于找第k个2和3的倍数，维护两个倍数值，其次根据数学思想，找 $ax+by=z$ ，实际上是找满足条件的最小x, y的公约数的倍数（这个我也不懂怎么来的，自己看推导）